

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1-INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA12	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO4	Assessment:	ASSESSMENT TOOL1-INDUSTRY PROBLEM SOLVING TASK
Weightage:	30%	Total Marks:	25
CO4 Statement:	<i>Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.</i>		
Student Name:		Reg.No.:	
Department:		Date:	

ANNEXURE A
INDUSTRY PROBLEM SOLVING TASK

- Smartphone Performance Optimization**
A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.
- Cloud Server Memory Bottleneck**
A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.
- Autonomous Vehicle Data Processing**
An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.
- AI Inference Edge Device**
An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.
- High-Performance Gaming Console**
A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

Code of Conduct:

I V. Reddy Sreya (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Understanding of Industry Problem (20%)	Application of Engineering Concepts (25%)	Solution Design (25%)	Use of Tools (15%)	Analysis (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

1. Smartphone Performance Optimization

Problem:

Lag occurs while switching between apps because frequently used instructions/data may not be available in fast cache memory, causing more accesses to slower DRAM.

Memory hierarchy:

CPU → L1 Cache → L2 Cache → L3 Cache → DRAM → Storage

Memory	Speed	Capacity	Role
L1	Fastest	Very small	Frequently used instructions/data
L2	Very fast	Small	Backup for L1
L3	Fast	Larger	Shared cache for CPU cores
DRAM	Slower	Large	Main memory for applications

Redesign:

- Increase **L1 cache** moderately for critical app data.
- Use a larger **L2 cache** to reduce accesses to L3.
- Increase **L3 cache** for shared application data.
- Use **high-bandwidth LPDDR5X/LPDDR6-class DRAM**.
- Apply intelligent cache replacement and prefetching.
- Keep recently used app states in memory instead of repeatedly loading them from storage.

Result:

More cache hits reduce memory access latency, while high-bandwidth DRAM improves throughput during multitasking. Therefore, app switching becomes faster and smoother.

2. Cloud Server Memory Bottleneck

Problem:

Database queries experience high latency because data may need to travel through multiple memory levels or be fetched from virtual memory/storage.

Cache hierarchy vs Virtual Memory Paging:

Feature	Cache Hierarchy	Virtual Memory Paging
Purpose	Reduce CPU memory-access latency	Extend available memory
Typical levels	L1, L2, L3	RAM + disk/SSD
Speed	Very high	Much slower when page faults occur
Unit	Cache line	Page
Main benefit	Faster repeated data access	Allows large applications

Recommended design:

CPU → L1 → L2 → L3 → DRAM → NVMe SSD

- Increase **L3 cache** for frequently accessed database indexes.
- Use sufficient **high-speed DDR5 DRAM**.
- Keep database buffer pools in DRAM.
- Use **NVMe SSD** rather than slower storage for paging and persistent data.
- Reduce page faults by allocating enough RAM.
- Use database-aware caching for frequently accessed records.
- Use huge pages where appropriate to reduce TLB overhead.

Conclusion:

Cache hierarchy should handle frequently accessed data, while virtual memory should be treated as a backup rather than the primary performance mechanism. More DRAM and efficient caching significantly reduce database latency.

3. Autonomous Vehicle Data Processing

An autonomous vehicle continuously processes camera, LiDAR, radar and other sensor data. Therefore, **low latency and high bandwidth** are critical.

Memory	Advantage	Limitation	Application
SRAM	Extremely fast, low latency	Expensive, low capacity	CPU/GPU caches
DRAM	High capacity and good bandwidth	Slower than SRAM	Sensor buffers, AI data
MRAM	Non-volatile, fast, low standby power	Higher cost/limited capacity	Critical persistent data

Proposed hierarchy:

CPU/GPU → SRAM → DRAM → MRAM → Flash/Storage

- **SRAM:** Store frequently accessed sensor-processing data and AI working sets.
- **DRAM:** Store large camera frames, LiDAR point clouds and intermediate AI tensors.
- **MRAM:** Store critical configuration, logs and data that must survive power loss.
- Use high-bandwidth DRAM and parallel memory channels.
- Use prefetching and DMA to transfer sensor data directly into memory.

Result:

SRAM minimizes latency for real-time computation, DRAM provides the required capacity and bandwidth, and MRAM provides fast non-volatile storage. This hierarchy supports real-time autonomous decision-making.

4. AI Inference Edge Device

The device needs to load AI models quickly while consuming little power.

Memory	Speed	Volatility	Main Use
NAND Flash	Moderate/slow	Non-volatile	Permanent model storage
DRAM	Very fast	Volatile	Active model execution
RRAM	Fast, non-volatile	Non-volatile	AI model/weight storage

Proposed hierarchy:

AI Accelerator/CPU → DRAM → RRAM → NAND Flash

- Store the complete AI model in **NAND Flash**.
- Keep frequently used model weights in **RRAM**.
- Load active model layers/tensors into **DRAM**.
- Use RRAM to reduce repeated transfers from NAND.
- Use model compression/quantization to reduce memory requirements.
- Use low-power DRAM and power-gating when memory blocks are idle.

Result:

NAND provides large-capacity persistent storage, RRAM provides fast and low-power non-volatile access, and DRAM supports active inference. This reduces model-loading time and power consumption.

5. High-Performance Gaming Console

Problem:

Large game scenes require large amounts of textures, models and other assets. If the CPU/GPU cannot obtain data quickly enough from DRAM, frame rates drop.

L3 Cache vs DRAM:

Feature	L3 Cache	DRAM
Latency	Very low	Higher
Capacity	Small	Large
Bandwidth	High	Very high
Purpose	Frequently reused data	Large game assets

Proposed hierarchy:

CPU/GPU → L1 → L2 → L3 → High-Bandwidth DRAM → NVMe SSD

Enhancements:

- Increase **L3 cache** to retain frequently reused game data.
- Use **high-bandwidth GDDR6/GDDR6-class memory or newer equivalent** for GPU workloads.
- Use multiple memory channels to increase bandwidth.
- Implement intelligent prefetching for upcoming scene assets.
- Stream assets from NVMe SSD before they are needed.
- Use compression to reduce the amount of data transferred.

Result:

A larger/faster L3 cache reduces repeated DRAM accesses, while high-bandwidth DRAM provides fast transfer of large textures and scene data. Together, they reduce frame drops and improve gaming performance.